ONLINE FOOD DELIVERY BUSINESS SYSTEM

[1]PROJECT TITLE

ONLINE FOOD DELIVERY BUSINESS SYSTEM

Submitted By-: Shefali Dixit
Course - SQL and AI
Mentor- Kalyani Bhatnagar

Project objective- In this project I used SQL to analyze the company's database and understand how the business is performing. The main goal of this project is to study revenue, customers, restaurants , and orders to find useful insights that can help improve the company's growth and performance.

Database Description -:

1. Customer table
   . Customer _ id
   . Customer _name
   . City
   .signup_ date

2. Order table
   . Order _id
   . Order _date
   . amount

3. Restaurant table
   . Restaurant _id
   . Restaurant _name
   . City

4. Order_ items table
   . Order_id
   . Items_name
   . quantity
   . price

SQL queries section -:

1. Total revenue

```
SELECT SUM( amount)As total_revenue
FROM orders;
```
This shows how many orders were placed in total.

2. Top 5 Customers

```
SELECT customer _id,SUM(amount) As total _spent.
FROM orders
GROUP BY customer _id
ORDER BY total_spent DESC
Limit 5;
```
This help identify customer who spend the most.

3. Order per Restaurant

```
SELECT restaurant _id, COUNT( order_id) As total_ order
FROM orders
GROUP BY restaurant_id;
```
This shows which restaurant receive more orders.

4. Monthly revenue

SELECT month(oder_date)
As month, SUM(amount)As revenue
FROM orders
GROUP BY month (order_date);
This helps analyse sales per month wise.

5. Average order value

SELECT avg(amount)As avg_order value.
FROM orders;
This shows the avg amount spent per order.

6. Customer from each city

SELECT city, count(cust_id) As total_customer.
FROM customers
GROUP BY city;
This shows which city has more customers.

7.Business insights

The company shows total revenue overall financial performance.
Some customers contribute more revenue than others.

8. Conclusion

In this project, I analysed the database of an online food delivery company using sql queries.
The analysis helped in understanding revenue, customer and rest personal . By using sql , meaning ful insights can be generated to improve business growth and operational efficiency.

Declaration:

I hereby declare that the project report is my original work. All source of information has been properly acknowledged.

Shefali Dixit
Date: 23 Feb 2026

Acknowledgement:

I would like to express my gratitude to my instructor for providing guidance and support throughout the development of this project. I also thanks to all online sources and learning platforms that helped me understanding sql concepts and implemented in this project successfully.

INDEX

1 . INTRODUCTION -:

The project focus on analyzing an online food delivery platform using structured relational data to derive meaning business insights. The dataset represents core operational entities.

2. Problem statement -:

Food delivery platforms face several operational challenges .
Identifying high value Customers

- Improving rest performance
- Reducing delivery time
- Increasing overall profitability
- Enhancing Customer satisfaction
  The objective of this project is to analyse historical food delivery data and generate actionable insights that helps improve business decision making.

3 . Business objectives

The key business objectives:

- Identifying top performing restaurant.
- Analyse Customer ordering behaviour.
- Segment customer based on purchase pattern.
- Evaluate Delivery performance and delays .
- Identify factors affecting revenue.
- Provide recommendations to improve efficiency and profit.

4. Dataset overview -:

Source of data
Number of rows and columns
Key column (Customer _id, order_id)
Data types.

5 . ER diagram (entity relations diagram).

Customer order
Order_restaurant
Order_delivery
Primary_key
Foreign _key
One to many relationships

6. Exploratory Data analysis (EDA)

- Data cleaning
- Remove duplicates
- Handle missing values
- Correct date types
- Descriptive Analysis
- Total revenue
- Total ordera
- Avg delivery time
- Popular restaurant
- Peak order time
- Visualization
- Revenue by month
- Order by city

7 . Customer Segmentation

- Order frequency
- Total spending

8. Restaurant performance Analysis

Revenue performance restaurant
Avg rating
Order volume
Preparation time
Cancellation rate

9. Delivery performance Analysis

- Avg delivery time
- Delayed orders
- Delivery partner efficiency
- Distance vs delivery time

10. View / Dashboard

Sql view
Dashboard include
Kpl's (revenue, order , delivery time)

11. Conclusion

Business impact
Future improvement.

The database structure of the online food delivery company consists of following tables:

1. **Customer Table**
   attributes:
   Cust_id (primary key)
   Customer _name
   City
   Phone no.
   Signup_date
   Each customer is uniquely identifies by cust_id .

2. **Restaurant Table**
   restaurant _id (primary key)
   Restaurant _name
   City
   Rating .
   Each restaurant is uniquely identifies by restaurant _id .

3. **Delivery_ agent table**
   agent_id(primary key)
   agent_name
   Phone no.
   City
   Each delivery agent is identified by agent_id .

4 . Order table
Order_id (primary key)
Customer _id(Foreign key)
Restaurant _id (Foreign key)
Agent_id (Foreign key)
Order_ date
Order_amount
Order_status
Each order is linked to one customer , one restaurant and one delivery agent.

5. Order_item table
Order_item id (primary key)
Order_id (foreign key)
Item_ name
Quantity
Price
One order have multiple items.

1. **Total Revenue**

   SELECT SUM(order_amount) As total_revenue.
   FROM orders;
   Total income generated from all orders.

2. **Total no. Of orders**

   SELECT COUNT(order_id)As total _orders
   FROM orders;
   This shows how many total orders were placed.

3. **Top 5 customer spending**

   SELECT customer _id (order_amount) As total _spent
   FROM orders
   GROUP BY customer _id
   ORDER BY total spent DESC
   Limit5;
   Insights- identify high value Customer.

4. **Order per Restaurant**

   SELECT restaurant _id, count(order_id) As total _orders
   FROM orders
   GROUP BY restaurant_id ;
   This shows restaurant performance based on no. Of orders.

5. **Top 5 restaurant by revenue.**

   SELECT restaurant _id, SUM (order_amount)As total_revenue
   FROM orders
   GROUP BY restaurant _id
   ORDER BY total_revenueDESC
   Limit 5;
   Best performing restaurant

6. **Repeat_customer(customer retention)**

   SELECT cust_id , cust (order_id) As order_count.
   FROM orders
   GROUP BY customer _id
   HAVING count(order_id)>1;
   Loyal customer who placed more than one order.

7. **Orders delivered by each agent**

   SELECT agent_id , COUNT (order_id) As total _deliveries
   FROM orders
   GROUP BY agent_id ;
   Analysed delivery agent performance.

8. **Average order value**

   SELECT Avg (order_amount) As average _order_value.
   FROM orders;
   Average spending per order.

9. **Most ordered items**

   SELECT item_name, SUM (quantity)As total quantity
   FROM order_items
   GROUP BY item_name
   ORDER BY total_quantity DESC ;
   The most popular food item

PHASE 1. Total Revenue is calculated by adding all orders amount.

1. SQL query

```
SELECT SUM(order_amount) As total_revenue.
FROM orders;
WHERE _order status = 'Delivered' ;
```

2. Total order per city

```
SELECT city, COUNT(order_id)As total_orders
FROM orders
GROUP BY city;
```

3. Top 10 Customer by spending

```
SELECT customer _name,SUM(order_amount) As total_spending
FROM orders
GROUP BY customer _name
ORDER BY total_spendingDESC
Limit 10;
```

PHASE 2. Customer Segmentation

Customer category (Gold/ silver/ Bronze)

```
SELECT cust_name,
SUM (order _amount) As total _spending,
CASE
WHEN SUM (order_amount)> 10000 Then 'GOLD'
WHEN SUM (order _amount) Between 5000AND 10000 Then 'silver'.
Else bronze
End As customer_category
FROM orders
GROUP BY customer _name;
```

PHASE3. Restaurant performance

1. Top 10 restaurant by revenue

```
SELECT restaurant _name,
SUM( order_amount) As revenue
FROM orders
GROUP BY restaurant _name
ORDER BY revenue _DESC
Limit 10;
```

2. Average rating vs revenue

```
SELECT restaurant _name,
AVG (rating) As avg _ rating,
SUM( order_amount) As revenue
FROM orders
GROUP BY restaurant _name;
```

PHASE 4. Delivery Analysis

1. Avg delivery time per city

```
SELECT city,
Avg ( delivery _time) As avg delivery time
FROM orders
GROUP BY city;
```

2. Late deliveries (above 45 min)

```
SELECT COUNT (order_id) As late deliveries.
FROM orders
WHERE delivery _time>45;
```

PHASE 5. Payment and discount analysis

1. Most use payment method

```
SELECT payment _method
COUNT( order_id ) As usage_count
FROM orders
GROUP BY payment _method
ORDER BY usage _count DESC;
```

2.
```
SELECT SUM (discount) As total_discount
FROM orders.
```

PHASE 6. Advanced SQL

1. Monthly Revenue using CTE
   CTE means common table expression
   It helps to write clean and readable queries.
   SELECT
   Month (order_date) As month,
   SUM (order_amount) As total _revenue
   FROM orders
   GROUP BY MONTH (order_date)
)
   SELECT* FROM monthly revenue
   This query calcultes total revenue for each month using CTE

2. Rank restaurant by revenue (window function)
   window function helps to rank data.
   SELECT
   Restaurant _name,
   SUM (order_amount)As revenue,
   Rank () OVER (order by sum (order_amount) DESC) As rank_ position.
   FROM orders
   GROUP BY restaurant _name;
   This query ranks restaurant based on their total revenue.

3. Above avg revenue (subquery)
   SELECT rest_name,SUM (order_amount)As revenue
   FROM orders
   GROUP BY restaurant _name
   HAVING SUM (order_amount)
   SELECT AVG(total revenue)
   SELECT SUM (order_amount) As total revenue.
   FROM orders
   GROUP BY restaurant _name
   AVG _table
   );
   This query shows rest earning more than avg revenue.

PHASE 7. Database object

1. **Create revenue view**

```
view is a virtual table
CREATE VIEW revenue _view As
SELECT restaurant _name,
SUM ( orders_amount)As revenue
FROM orders
GROUP BY restaurant _name;
SELECT* FROM revenue _view;
```

2. **Stored procedure - Get top10 restaurant**

```
CREATE PROCEDURE get top restaurant (IN TOP _BEGIN)
SELECT restaurant _name;
SUM (order_amount)As revenue
FROM orders
GROUP BY restaurant _name
ORDER BY revenue DESC
Limit top 10;
End;
Call get top rest (10);
```

PHASE 8. Performance optimization

```
Index on order_date
CREATE INDEX. Idx _order_date
On order ( order_date);

Index on customer _id
CREATE Index .idx _customer_id
On orders(cust_id);

Index on rest_id
CREATE INDEX.idx _restaurant_id
On orders( rest_id);
```

PHASE 9. Automation logic

1. **Triggers - prevent Negative Discount**

```
CREATE tigger check_ discount
Before INSERT ON orders
For each row
BEGIN
If new.discount < 0 then
Set new . discount=0;
End if;
End ;
```

2. **Triggers - Delivery delay warning**

```
CREATE TRIGGER delivery 0_lwarning
Before INSERT ON orders
For each row
Begin
If new. Delivery _time> 60then
Signal SQL state' 45000'
Set message _text= ' Delivery time too long ' ;
End if;
End ;
```

PHASE 10. Final conclusion

In this project, advanced sql like CTE , window function, indexes and trigger were implemented. These techniques improve performance, automation and data mgmt efficiency.